Teoria essencial para a defesa do RAG

O que precisas de saber e o porquê, em poucas linhas, ligado ao teu projeto

1. RAG em 3 frases

Um LLM pequeno não sabe o código do vLLM e inventa quando não sabe (**alucinação**). RAG (Retrieval-Augmented Generation) resolve isto em dois passos: **1)** procura os trechos relevantes numa base de documentos; **2)** dá esses trechos ao LLM como contexto e manda-o responder só com eles. **Porquê:** respostas baseadas em fontes verificáveis, sem retreinar o modelo e com conhecimento atualizável.

2. O teu pipeline numa linha

> vLLM (.py, .md...) -> **chunking** (AST / parágrafos, máx. 2000) -> **tokenize** -> **BM25** guardado em disco. Pergunta -> tokenize -> scores BM25 -> **top-k** (ficheiro + offsets) -> relê o texto das 3 melhores -> **prompt** -> Qwen3-0.6B -> resposta.

3. Chunking: o que é e porquê

- **O que é:** partir os documentos em pedaços para indexar e recuperar. Documentos inteiros são grandes demais e diluem a relevância.
- **Pequenos demais:** perdem contexto e têm poucos termos para pontuar. **Grandes demais:** ruído, diluem o sinal e não cabem no contexto do LLM. 2000 caracteres é o compromisso pedido.
- **Porque 2 estratégias:** código tem estrutura (funções, classes), por isso cortar por AST mantém cada função inteira; texto e markdown cortam-se por parágrafos. Cortar a meio de uma função estragaria o significado.
- **Porque guardas offsets e não texto:** o índice fica leve, a avaliação compara intervalos de caracteres e o texto relê-se do ficheiro só quando é preciso.

4. Tokenização e BM25

- **Tokenizar:** transformar texto em palavras comparáveis (minúsculas, sem pontuação). O teu também parte snake_case em partes (configure_openai_server dá configure, openai, server) porque o código usa muitos identificadores compostos.
- **TF-IDF:** uma palavra pesa mais se aparece muito no documento (TF) e pouco no conjunto (IDF, é rara e informativa).
- **BM25 = TF-IDF melhorado:** (1) **saturação**: repetir uma palavra 50 vezes não vale 50x mais (parâmetro `k1`); (2) **normalização pelo comprimento**: chunks longos não ganham só por serem longos (parâmetro `b`). Valores habituais: k1 = 1.5, b = 0.75.
- **Porque BM25:** rápido, sem GPU, sem treino, bom em termos exatos como nomes de funções e flags, que é o que o código tem. **Limite:** não percebe sinónimos nem significado ("car" não encontra "automobile").
- **Embeddings (alternativa):** transformam texto em vetores e comparam significado, apanhando sinónimos, mas precisam de modelo e são mais lentos. **Híbrido** = combinar os dois.

5. Avaliação: recall, k e IoU

- **Recall@k:** das fontes certas, que fração aparece nos k primeiros resultados. Sobe com k, mas k grande traz mais ruído. Por isso medes @5 e recuperas até 10.
- **Precision@k** (para contraste): dos k devolvidos, quantos são certos. Aqui o recall é o que importa mais: o LLM consegue ignorar algum ruído, mas não consegue usar uma fonte que não foi recuperada.
- **IoU** (Intersection over Union): sobreposição entre dois intervalos = tamanho da interseção a dividir pelo da união. Os chunks nunca coincidem exatamente com a fonte certa, por isso conta como encontrada se a sobreposição passar um limiar (no teu evaluate, 0.05, e tem de ser o mesmo ficheiro).
- **Thresholds da avaliação:** docs pelo menos 0.80 e code pelo menos 0.50 de Recall@5. Código é mais difícil: vocabulário parecido entre funções e perguntas mais abstratas.

6. Geração com o LLM

- **Qwen3-0.6B:** o subject impõe-no. É pequeno, corre em CPU, mas é fraco, por isso depende muito de um bom contexto.
- **Janela de contexto limitada:** por isso usas só as 3 melhores fontes e no máximo 2000 caracteres, mesmo com k=10.
- **Prompt:** instrui o modelo a usar só o contexto e a dizer que não sabe se não houver informação. É a principal defesa contra alucinação.
- **Geração sem amostragem (greedy):** escolhe sempre a palavra mais provável, dá respostas estáveis e repetíveis. `repetition_penalty` e `no_repeat_ngram` evitam que um modelo pequeno entre em ciclos.
- **enable_thinking=False:** desliga o raciocínio longo do Qwen3, mais rápido e com respostas diretas.

7. Porque estas ferramentas

Ferramenta	Porquê
Pydantic	Valida a estrutura dos JSON (perguntas, fontes, respostas). Erros de formato falham cedo e com mensagem clara.
Fire	Transforma os métodos da classe RAGSystem em comandos de terminal sem escrever argparse.
uv + pyproject + uv.lock	Ambiente reproduzível: toda a gente instala as mesmas versões.
tqdm	Barra de progresso nas operações longas (indexar, pesquisar o dataset).
flake8 / mypy	Qualidade de código: estilo e verificação de tipos.
Índice em disco (pickle)	Indexa-se uma vez e reutiliza-se em cada comando, sem repetir o trabalho.

8. Perguntas-armadilha, respostas curtas

Pergunta	Resposta curta
Porque BM25 e não embeddings?	Rápido, sem GPU e ótimo em termos exatos de código. O custo é não apanhar sinónimos; um híbrido seria a melhoria.
O que acontece se a pesquisa falhar?	O LLM recebe contexto errado e responde mal ou diz que não sabe. Por isso se mede o retrieval separadamente.
O que muda se aumentares o k?	Recall sobe, mas mais ruído. O gerador só usa 3 fontes de qualquer forma.
E se a query não tiver resultados?	BM25 dá score 0, descartam-se esses e imprime "No results found." sem erro.
Porque código tem recall mais baixo?	Perguntas abstratas contra vocabulário técnico repetido entre funções; as palavras da pergunta nem sempre aparecem na fonte.
Como melhorarias?	Pesquisa híbrida com embeddings, query expansion, reranking, e pedir ao modelo que cite as fontes.

9. Os teus pontos fracos (assume-os antes de te perguntarem)

1) O prompt não manda citar as fontes e o `answer` não as imprime. **2)** A separação de camelCase não funciona porque o texto passa para minúsculas antes dela; snake_case funciona. **3)** Só 3 fontes e 2000 caracteres chegam ao LLM. **4)** Não tens bónus implementado. Dizer isto com naturalidade e com a melhoria que farias mostra que percebes o sistema.